

pipeline_lab — Referência Completa

Funções, parâmetros e fundamentação na literatura

modelagem-lab

12 de julho de 2026

Resumo

Este documento cataloga toda função pública de `pipeline_lab` ([python/pipeline_lab/](#) no repositório), seus parâmetros, e a passagem de literatura que justifica cada decisão de design relevante. A biblioteca é uma coleção de funções soltas (não uma API fluente) para montar o funil de modelagem de risco de crédito — divisão, construção de variáveis, agregação temporal, descoberta de interação, categorização/WOE, pré-seleção e a busca stepwise `Pedro_Wise` — em cima de um `pandas.DataFrame` qualquer, sem depender de disco ou de um servidor web. As citações vêm da base de literatura já curada em `docs/literatura/` do repositório.

Sumário

1	Filosofia e convenções	2
2	divisao	2
3	construcao	3
4	esfera1 — agregação temporal	3
5	esfera2 — descoberta de interação	4
6	categorizar — binning + WOE	5
7	preselecao	6
8	treinamento — Pedro_Wise	7

1 Filosofia e convenções

`pipeline_lab` é uma coleção de **funções soltas**, não uma API orientada a objeto: cada etapa recebe `DataFrames` e devolve `DataFrames` novos, nunca muta o original, nunca lê ou escreve em disco. Isso permite plugar um `DataFrame` qualquer e escolher só os módulos que fazem sentido para o caso de uso, na ordem:

```
divisao -> construcao (opcional) -> esfera1 (opcional)
        -> esfera2 (opcional) -> categorizar
        -> preselecao (opcional) -> treinamento
```

Convenção de coluna-alvo. A partir de `divisao`, a variável resposta deve se chamar "y" em `df_dev/df_teste` — todas as etapas seguintes acessam `df["y"]` diretamente. `dividir_por_amostra/dividir_aleatorio` aceitam `coluna_y` para fazer esse rename automaticamente.

Anti-leakage por construção. Toda etapa que "ajusta" algo a partir de y (WOE, tabela de bins monotônicos, regras de interação) faz isso só em dev, e reaplica a mesma tabela/regra em teste sem reajustar — nunca recalcula usando o y de teste. É o mesmo padrão *fit/transform* do scikit-learn.

Convenção de "base". Uma variável crua pode ter várias versões (`idade_woe`, `idade_log`, `idade_bin`) — todas compartilham o mesmo prefixo semântico ("base"). O `Pedro_Wise` nunca deixa duas versões da mesma base coexistirem no modelo final.

2 divisao

Divide um `DataFrame` em (`df_dev`, `df_teste`). Sempre o primeiro passo.

`dividir_por_amostra(df, coluna_amostra, valores_dev, valores_teste, coluna_y=None)`

Split por uma coluna de amostra que já existe no dataframe — não assume nomes fixos: você diz quais rótulos contam como dev e quais contam como teste.

Parâmetro	Tipo	Default	Explicação
<code>df</code>	<code>DataFrame</code>	—	Dataframe completo, ainda não dividido.
<code>coluna_amostra</code>	<code>str</code>	—	Coluna que identifica a amostra de cada linha (ex. "split"). Valores livres — "DES"/"OOT", "train"/"val", o que já vier na fonte.
<code>valores_dev</code>	<code>list[str]</code>	—	Quais valores contam como desenvolvimento.
<code>valores_teste</code>	<code>list[str]</code>	—	Quais valores contam como teste. Linhas fora das duas listas são descartadas.
<code>coluna_y</code>	<code>str None</code>	<code>None</code>	Se passado, renomeia essa coluna para "y" antes de dividir.

`dividir_aleatorio(df, proporcao_teste=0.5, semente=42, coluna_y=None)`

Split aleatório simples — embaralha e corta.

Parâmetro	Tipo	Default	Explicação
<code>proporcao_teste</code>	float	0.5	Fração das linhas que vai para teste.
<code>semente</code>	int	42	<code>random_state</code> — reprodutibilidade.
<code>coluna_y</code>	str None	None	Ver acima.

3 construação

Razões e diferenças interpretáveis entre pares de variáveis relacionadas — ex. `pago / fatura =` "proporção paga da fatura", uma feature de negócio óbvia que não existe nas colunas originais.

Escopo deliberadamente mínimo: não é busca automática tipo Deep Feature Synthesis ou programação genética. A escolha de `v1` foi gerar candidatas simples e interpretáveis primeiro — mais barato e testável imediatamente contra o Pedro_Wise — em vez da escola "automática/exaustiva" descrita no paper fundador de Kanter & Veeramachaneni (2015), *Deep Feature Synthesis: Towards automating data science endeavors*, que aplica operações de agregação recursivamente sobre relações entre tabelas (base da biblioteca `featuretools`).

Um paralelo direto em crédito: Dumitrescu et al. (2021), *Machine learning for credit scoring: Improving logistic regression with non-linear decision-tree effects*, usa efeitos extraídos de árvores como features construídas de volta para uma regressão logística — o mesmo espírito de "construção assistida" que `esfera2` (RuleFit-style) aplica depois de `construcao` no funil.

Função	Assinatura e parâmetros
<code>construir_razao</code>	(numerador, denominador, nome, <code>epsilon=1e-6</code>) — <code>epsilon</code> substitui denominador exatamente zero (mantém o sinal no resto, só evita divisão por zero literal); comum em variáveis monetárias reais.
<code>construir_diferenca</code>	(a, b, nome) — a - b, útil para variáveis na mesma unidade (ex. variação de fatura mês a mês).
<code>construir_razoes_em_log</code>	(df, pares, <code>epsilon=1e-6</code>) — <code>pares</code> é uma lista de (numerador, denominador, nome), aplica <code>construir_razao</code> para cada tripla. Retorna só as colunas construídas.

4 esfera1 — agregação temporal

Agregação "comportamental" (behavioral scoring): a partir de um painel (chave + tempo + variável observada mês a mês), constrói primitivas de janela móvel (máximo, média, mínimo, desvio-padrão, tendência) e devolve uma linha por chave.

Catálogo de primitivas inspirado no vocabulário de agregação do Deep Feature Synthesis (Kanter & Veeramachaneni, 2015) — sem adotar a biblioteca inteira, só o vocabulário relevante para painéis de crédito mensal.

Garantia sem look-ahead: cada linha de saída usa só observações até e incluindo o próprio período — nunca dados futuros da mesma chave.

Preservação de split: aplicar agrega `df_dev` e `df_teste` *separadamente* — nunca junta as duas bases para agregar e depois redividir, o que contaminaria a fronteira temporal do split.

Função	Assinatura e parâmetros
<code>agregar</code>	<code>(df, chave, coluna_tempo, colunas_valor, janelas)</code> — core de agregação sem split, uma chamada por base. <code>chave</code> identifica a entidade; <code>coluna_tempo</code> deve ser ordenável por chave; <code>colunas_valor</code> são as colunas numéricas agregadas; <code>janelas</code> é uma lista de tamanhos de janela móvel em períodos (ex. <code>[3, 6, 12]</code>). Retorna <code>(df_agregado, colunas_geradas)</code> .
<code>aplicar</code>	<code>(df_dev, df_teste, chave, coluna_tempo, colunas_valor, janelas)</code> — chama <code>agregar</code> separadamente em dev e teste. Retorna <code>(df_dev_agregado, df_teste_agregado, colunas_geradas)</code> .

5 esfera2 — descoberta de interação

Descoberta de interação RuleFit-style (Friedman & Popescu, 2008, *Predictive learning via rule ensembles, Annals of Applied Statistics*): treina um ensemble de árvores rasas (`GradientBoostingClassifier`) sobre as candidatas já construídas, extrai os caminhos raiz-folha como regras de interação (ex. "tendência alta E severidade alta"), e devolve as regras como candidatas avaliáveis para o funil.

Diferença deliberada do RuleFit original: no paper, as regras extraídas viram termos de uma regressão com penalização L1 (Lasso) que decide os pesos finais. Aqui as regras viram colunas 0/1 que entram no funil normal — quem decide o que fica no modelo final continua sendo a busca stepwise do `Pedro_Wise`, não uma regressão L1 própria.

`transformar_categoricas_woe(dev, teste, colunas_x, colunas_categoricas=None)`

O `GradientBoostingClassifier` só aceita entrada numérica, então colunas categóricas são WOE-codificadas antes de entrar na árvore. Colunas de texto em `colunas_x` são tratadas como categóricas automaticamente. É um encoding "ingênuo" (sem regularização/shrinkage) — Pargent, Pfisterer, Thomas & Bischl (2022), *Regularized target encoding outperforms traditional methods...*, mostra que versões regularizadas de target encoding superam a versão ingênua, que pode vaziar informação do target e overfittar; ponto de atenção registrado, ainda não implementado aqui (o WOE final que o modelo vê é ajustado só em dev, o mesmo cuidado anti-leakage do resto do lab).

`aplicar(df_dev, df_teste, ...)`

Parâmetro	Tipo	Default	Explicação
<code>colunas_categoricas</code>	<code>list[str] None</code>	<code>None</code>	Ver <code>transformar_categoricas_woe</code> .
<code>profundidade_maxima</code>	<code>int</code>	<code>2</code>	Profundidade das árvores — controla o tamanho máximo de uma regra (nº de condições).
<code>n_arvores</code>	<code>int</code>	<code>60</code>	Nº de árvores no GBM — mais árvores, mais candidatas de regra antes da deduplicação/filtro.

Parâmetro	Tipo	Default	Explicação
<code>min_suporte</code>	float	0.02	Suporte mínimo (fração de linhas que satisfaz a regra) — descarta regras quase nunca verdadeiras.
<code>max_suporte</code>	float	0.5	Suporte máximo — descarta regras quase sempre verdadeiras (constante disfarçada).
<code>max_regras</code>	int	10	Teto de regras devolvidas após os filtros.
<code>permitir_cruzamento_entre_bases</code>	bool	True	Se False, restringe regras a combinarem só primitivas da mesma variável bruta — útil quando não se quer misturar domínios numa condição só.
<code>proporcao_variaveis_por_coluna</code>	float	None	Fração de variáveis candidatas por split (equivalente a <code>max_features</code> do sklearn) — diversifica as regras extraídas.
<code>iv_minimo</code>	float	0.02	Só regras com IV de <i>teste</i> $\geq$ esse valor viram coluna — nunca o IV de dev, que já foi usado para escolher a regra e está inflado por construção.

Retorna (df_dev, df_teste, colunas_geradas).

6 categorizar — binning + WOE

Sempre a última etapa antes da pré-seleção/treinamento — é aqui que as versões alternativas de cada variável (`_woe`, `_log`, `_bin`) nascem. Esfera 2 vem antes justamente porque essas transformações ainda não existem naquele ponto do funil.

```
categorizar_e_transformar(df_dev, df_teste, gerar_transformacoes_potencia=True,
gerar_bin_ordinal=True, ao_processar_coluna=None)
```

Para cada coluna (exceto `y`):

- **Numérica contínua** (`nunique > 2`): binning monotônico primeiro (`bins_monotonicos` — merge guloso que força a taxa de evento a ser monotônica entre bins adjacentes; versão pragmática, não ótima via MIP, da ideia central do OptBinning — Navas-Palencia, 2020, *Optimal binning: mathematical programming formulation*, hoje o padrão prático de mercado para binning em crédito).
- **Catégorica/binária** (texto, ou numérica com só 2 valores — ex. uma flag 0/1 de regra da Esfera 2): sem binning, cada valor já é o "bin". *Bug real corrigido*: colunas binárias eram descartadas silenciosamente aqui antes da correção de 2026-07 (`bins_monotonicos` numa coluna de 2 valores gera só 2 edges, insuficiente para discretizar).

Depois do binning: `ajustar_woe` (Siddiqi, *Credit Risk Scorecards* — convenção de sinal $WOE = \ln(\%no - evento / \%evento)$, positivo quando o bin tem mais "bons pagadores" relativo à distribuição total) ajustado só em dev, reaplicado em teste.

Parâmetro	Tipo	Default	Explicação
<code>gerar_transformacoes_potencia</code>	<code>bool</code>	<code>True</code>	Gera também <code>log/raiz/quad/cubo/inversas</code> como candidatas extras — o <code>Pedro_Wise</code> testa trocar a versão <code>WOE</code> por uma dessas via a semântica de <code>"base"</code> . Só numérica contínua; descarta transformações com domínio inválido. Análogo em espírito a <code>Box-Cox/Yeo-Johnson</code> (Atkinson, Riani & Corbellini, 2021).
<code>gerar_bin_ordinal</code>	<code>bool</code>	<code>True</code>	Gera também o índice do bin (faixa) como candidata extra — outra "versão" da mesma base.
<code>ao_processar_coluna</code>	<code>Callable None</code>	<code>None</code>	Callback (<code>coluna</code> , <code>iv</code>) chamado após cada coluna processada — gancho de progresso em tempo real, sem a biblioteca precisar saber o que é uma fila/WebSocket.

Retorna (`woe_dev`, `woe_teste`, `iv_por_variavel`).

7 preselecao

Reduz o volume de candidatas antes do `Pedro_Wise`. Motivação: depois que Construção e as transformações de potência de `categorizar` geram dezenas/centenas de colunas extras por variável-base, o volume pode explodir — mais lento e maior risco de selecionar ruído por acaso (o problema de múltiplos testes, o mesmo pano de fundo da literatura de *stability selection*: Meinshausen & Bühlmann, 2008/2010, *Stability Selection*, JRSS-B).

Três filtros em sequência, cada um opcional (`limiar=None` pula):

Função	Parâmetros e explicação
<code>filtrar_variancia(df, colunas, limiar=1e-6)</code>	Mantém colunas com variância $> \text{limiar}$ — descarta quase-constantes, comuns entre transformações de potência quando a variável original tem pouca variação.
<code>filtrar_iv(colunas, iv_por_base, limiar=0.02)</code>	Mantém colunas cuja variável-base tem $IV \geq \text{limiar}$. Bases sem entrada em <code>iv_por_base</code> são tratadas como $IV=0$ (descartadas).
<code>filtrar_correlacao(df, colunas, iv_por_base, limiar=0.9)</code>	Entre pares com $ r \geq \text{limiar}$, mantém o de maior IV . Pares da mesma base são ignorados de propósito — filtrar aqui colapsaria a família inteira num só sobrevivente antes do <code>Pedro_Wise</code> escolher a melhor versão.
<code>pre_selecionar(df, iv_por_base, limiar_variancia=1e-6, limiar_iv=0.02, limiar_correlacao=0.9)</code>	Aplica os 3 filtros em sequência. Retorna <code>colunas_mantidas</code> , contagens em cada etapa do funil, e pares descartados por correlação.

Este design — filtrar correlação entre bases diferentes, nunca dentro da mesma base — evita o modo de falha descrito por Faletto & Bien (2022), *Cluster Stability Selection*: quando há proxies altamente correlacionados de um fator latente, um filtro de correlação ingênuo pode escolher arbitrariamente um proxy e nunca convergir no correto. A distinção "mesma base nunca filtra, base diferente filtra" é a fronteira que evita esse problema por construção.

8 treinamento — Pedro_Wise

O "famigerado" Pedro_Wise — busca greedy multi-nível (forward/backward, níveis 1 a 3) sobre GLM binomial (logística), otimizando KS (estatística de Kolmogorov–Smirnov entre a distribuição de score de eventos e não-eventos). Port fiel de um script R original, validado numericamente contra o R (KS-dev bate em 10 casas decimais).

```
treinar(df_dev, df_teste, ...)
```

Todas as colunas de `df_dev`/`df_teste` exceto `y` são candidatas.

Parâmetro	Tipo	Default	Explicação
<code>criterio</code>	<code>teste dev min(teste)</code>		Não é cosmético — é o que forward/backward otimiza em toda a busca. O R original tem um bug que decide só por KS-dev; a validação do port mostrou que sob <code>criterio="dev"</code> o algoritmo aceita uma variável de puro ruído por acaso amostral. <code>"teste"</code> é o default recomendado para uso real.
<code>shadow_probing</code>	<code>bool</code>	<code>False</code>	Reservado para um critério de parada alternativo (backlog) inspirado em Thomas, Hepp, Mayr & Bischl (2017), <i>Probing for Sparse and Fast Variable Selection with Model-Based Boosting</i> : aumentar as candidatas com cópias permutadas ("shadow variables") e parar quando uma shadow seria selecionada — sinal de ajuste a ruído, sem precisar separar dev/teste para decidir o ponto de parada.
<code>forward_simples</code>	<code>bool</code>	<code>True</code>	Nível 1: testa adicionar cada variável fora do modelo, uma por vez.
<code>transformacao_simples_nivel1</code>	<code>bool</code>	<code>True</code>	Nível 1: troca a versão de uma base já no modelo por outra versão da mesma base.
<code>backward_simples_nivel1</code>	<code>bool</code>	<code>True</code>	Nível 1: remove uma variável por vez, se o KS melhora.
<code>min_vars_para_backward</code>	<code>int</code>	<code>5</code>	Backward só é tentado acima desse número de variáveis no modelo.

Parâmetro	Tipo	Default	Explicação
<code>forward_duplo</code>	bool	True	Nível 2: testa pares de variáveis do <code>top-n_best_duplo</code> do forward simples — pega sinergias que forward variável-a-variável não descobre sozinho.
<code>forward_triplo</code>	bool	True	Nível 2.5: combina <code>top-n_best_triplo_1</code> × <code>n_best_triplo_2</code> do forward duplo em triplas.
<code>transformacao_simples_nivel2</code>	bool	True	Mesma troca de versão de base, repetida no nível 2.
<code>backward_simples_nivel2</code>	bool	True	Mesmo backward, repetido no nível 2.
<code>n_best_duplo</code>	int	5	Quantas variáveis do forward simples entram na busca de pares.
<code>n_best_triplo_1</code> , <code>n_best_triplo_2</code>	int	3, 3	Quantas variáveis de cada lado entram na busca de triplas.
<code>nivel3_ativado</code>	bool	False	Ativa o backward complexo (nível 3): remove e re-roda o <code>Pedro_Wise</code> inteiro a partir dali (recursão). Maior custo/risco combinatorial.
<code>n_best_backward</code>	int	2	Quantas variáveis avalia para remoção no nível 3.
<code>profundidade_maxima_nivel3</code>	int	2	Limite de profundidade da recursão do nível 3.
<code>p_valor_maximo</code>	float None	None	Filtro opcional de significância — não é o critério de seleção (o <code>Pedro_Wise</code> nunca usa p-valor para decidir, sempre KS).

Retorna `ResultadoTreinamento`: `variaveis`, `ks_dev`, `ks_teste`, `auc_teste`, `taxa_evento_dev`, `taxa_evento_teste`, `coeficientes`, `estatisticas` (diagnóstico pós-hoc, não influencia a seleção), `tabela_decis`.

Bibliografia citada

Friedman, J. H., & Popescu, B. E. (2008). *Predictive learning via rule ensembles*. Annals of Applied Statistics. — base de `esfera2`.

Kanter, J. M., & Veeramachaneni, K. (2015). *Deep feature synthesis: Towards automating data science endeavors*. — vocabulário de primitivas de `construcao/esfera1`.

Navas-Palencia, G. (2020). *Optimal binning: mathematical programming formulation*. arXiv:2001.08025. — inspiração de `bins_monotonicos` em `categorizar`.

Siddiqi, N. *Credit Risk Scorecards*. — convenção de WOE em `categorizar`.

Meinshausen, N., & Bühlmann, P. (2008/2010). *Stability Selection*. JRSS-B. — motivação de `preselecao`.

Faletto, G., & Bien, J. (2022). *Cluster Stability Selection*. — desenho de `filtrar_correlacao` (exceção para mesma base).

Thomas, J., Hepp, T., Mayr, A., & Bischl, B. (2017). *Probing for Sparse and Fast Variable Selection with Model-Based Boosting*. Computational and Mathematical Methods in Medicine. — `shadow_probing` em `treinamento` (backlog).

Pargent, F., Pfisterer, F., Thomas, J., & Bischl, B. (2022). *Regularized target encoding outperforms traditional methods...* — nota de cautela em `transformar_categoricas_woe`.

Atkinson, A. C., Riani, M., & Corbellini, A. (2021). *The Box-Cox Transformation: Review and Extensions*. — paralelo às transformações de potência de `categorizar`.

Ver `docs/literatura/` no repositório para a lista completa com links e sínteses mais longas por tópico.